# Semantic Textual Similarity

Introduction to Human Language Technologies

Óscar Cubeles & Valentin Soumah
Master in Artificial Inteligence
2025-26

# Table of Contents


1. **Methodology**
2. **Features**
    a. Match Based Features
    b. Lexical Features
    c. Syntactical Features
3. **Baseline Model Evaluation**
4. **Feature Selection**
5. **Results on Test Set**
6. **Final Takeaways**

# Methodology

Training Data



Preprocessing [1]

- Remove special characters
  - **-, /, > & <**
- Standarize dollar format:
  - **$US** to **$**
- Replace contractions.
  - e.g: ***n't*** with ***not***

Feature Extraction

- Match Based Features
- Lexical Features
- Syntactical Features

Baseline Model Evaluation

- Ridge Regression
- Random Forest
- SVR

Feature Selection

- Backward Feature Elimination
- Cross-Model Feature Interpretation

Model Evaluation on Test Set

# Match Based Features

**1. Word Unigram distance, Bigram and Trigram Jaccard Similarity:**

Measure how many word, bigrams or trigrams the pairs of sentences share after removing stopwords.

```
w_unigrams("The quick brown fox jumps", "The fast brown fox leaps")  = [“brown”, “fox”]
w_bigrams(“"The quick brown fox jumps", "The fast brown fox leaps") = [”brown fox”]
w_trigrams(“"The quick brown fox jumps", "The fast brown fox leaps") =  None
```

**2. Dice Number Similarity:**[1]

Measures how many numbers two texts share using Dice similarity to give more importance to intersections.

```
num_sim(“Today the parliment is missing 3 members.”, “The parliament misses three or four members ”) = 0.66
```

**3. Date Similarity:**

Measures how close the dates mentioned in two texts are, scoring higher for exact or near matches and decaying exponentially as the time difference increases.

```
date_sim(“The meeting is scheduled for 12/05/2023.”, “We will meet on May 2023”) = 0.8479
```

# Lexical Features

**1. Lemma Unigram Distance, Bigram and Trigram Jaccard Similarity:**

Measure how many lemmas, lemma-bigrams or trigrams the pairs of sentences share after removing stopwords.

```
l_unigrams("The quick brown fox is jumping", "The fast brown fox jumps")  = ["brown", "fox", "jump"]
l_bigrams("The quick brown fox is jumping", "The fast brown fox jumps")   = ["brown fox"]
l_trigrams("The quick brown fox is jumping", "The fast brown fox jumps")  = None
```

**2. Synset Jaccard Distance:**

Measures sentence similarity based on the overlap of word senses (synsets) derived using WSD.

```
synset_sim(“The animal attacked the person“, “The beast charged the individual”) = 0.8
```

**3. Weighted WordNet Similarity and Sentence Rarity Similarity:**

Features that Measure word to word sentence similarity weighting words by rarity (brown corpus) and WordNet depth.

```
r_sim("The quantum experiment was successful.", "The experiment on quantum physics was a success.")=0.505
```

[1]

**4. Sentence Path, LCH, WUP, and Lin Similarity (Mean & Max)**

Each word in one sentence is matched to its most similar word in the other sentence in terms of Path Similarity, Lin Similarity Wup Similarity and LCH similarity. The overall sentence similarity is computed by aggregating these maximum word similarities, using either the mean or the max.

# Syntactical Features

**1. POS Jaccard Distance and POS Levenshtein Distance**
Measure similarity based on overlap of **POS tags.** Levenshtein distance measures the minimum edits needed to transform one POS sequence into another.

**2. Dependency Label Distance:**
Measures similarity by computing the Levenshtein distance between sequences of dependency labels (`nsubj`, `obj`, `ROOT`, etc.)

**3. LCS, LCSubstr & Greedy String Tiling:**[2]
Capture how much two sentences word order. **LCS** is the longest sequence of not necessaryily contiguous words appearing in order. **LCSubstr** is the longest contiguous sequence of words, and **Greedy String Tiling** finds the longest non-overlapping matching token sequences above a minimum length.

**4. Named Entity Presence and Named Entity Distance** [1]
Indicate whether sentences contain named entities **(presence)** and measure similarity of entity types between sentences using Jaccard **distance**

**5. Sentence Length similarity** [1]
Absolute difference in sentence lengths divided by the maximum length

# Baseline Model Evaluation

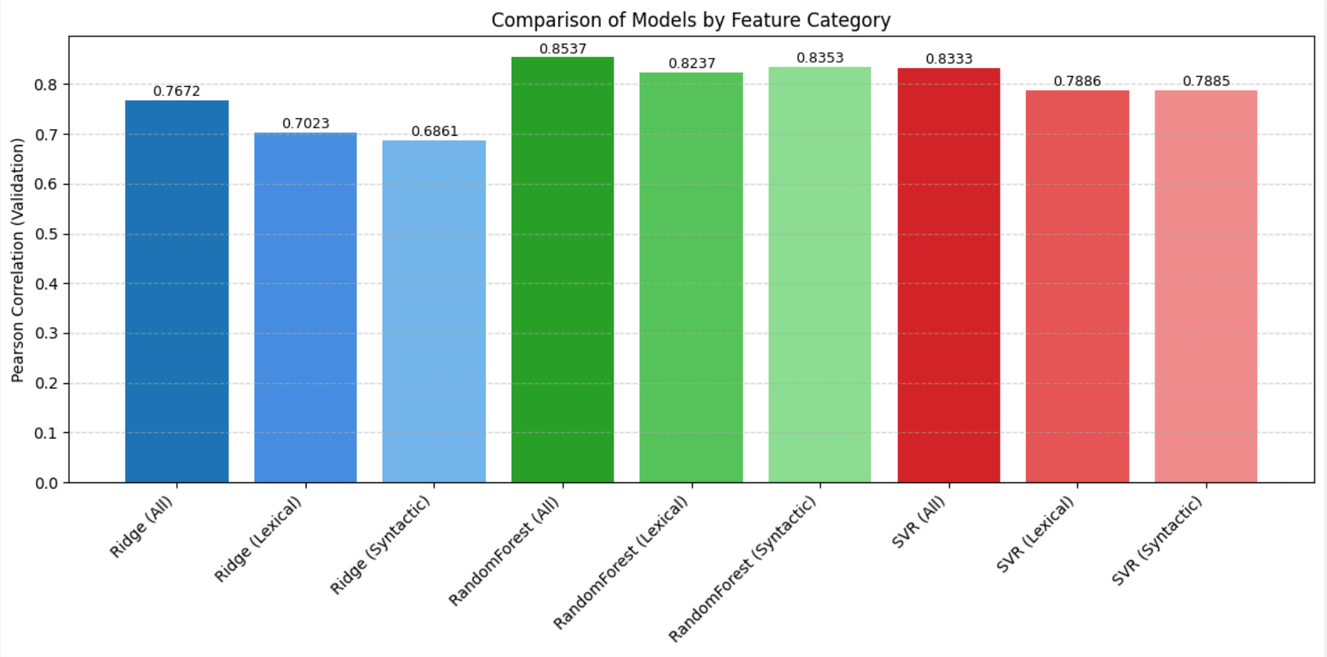
## On the validation set

**Models:**

- **Ridge Regression:** A simple linear model
- **Random Forest Regression:** An ensemble model that captures non-linear interactions in the features.
- **Support Vector Regression (SVR):** SVM-based model

**Feature Sets:**

- **Lexical Features**
- **Syntactical Features**
- **All Featrues:** Lexical + Syntactical + Match-Based



# Feature Selection

**Backwards Elimination:**

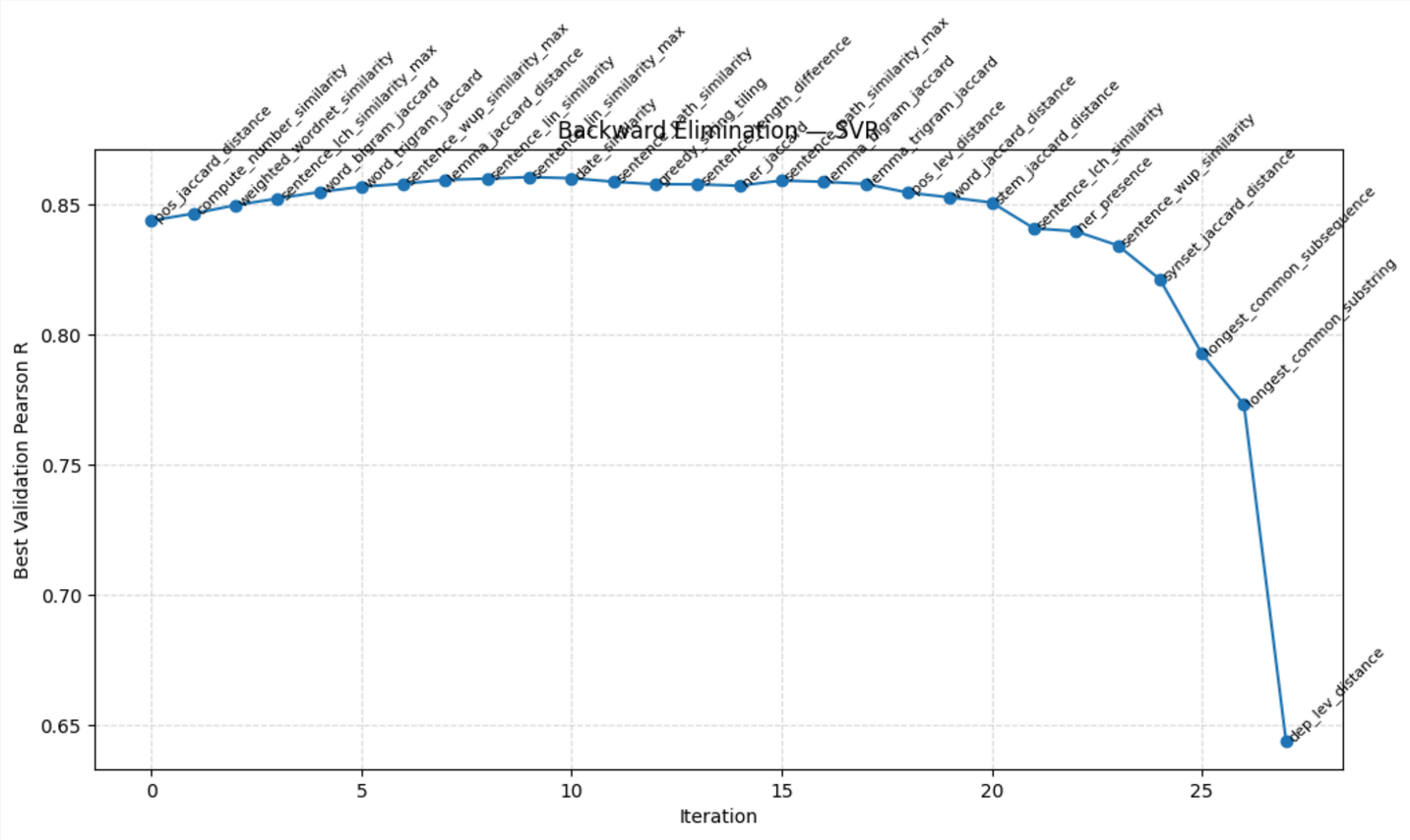
- Removes worst performing feature at each iteration
- Performs cross validation to find best model for given feature subset
- Computes validation r for each subset

**Feature importances:**

- **Ridge Regression Coefficient :** straighfoward linear importance
- **Random Forest Feature importance :** permutations

**Feature Correlation Matrix**

- Identifying features that bring the same information.
- Deciding which features to remove




Backward elimination history of SVR

# Eliminated Features

**Wordnet similarity metrics** : *sentence_lin_similarity, sentence_lin_similarity_max  sentence_path_similarity  sentence_lch_similarity, sentence_wup_similarity*

**Jaccard n-gram word metrics:** *word_bigram_jaccard, word_trigram_jaccard`, lemma_bigram_jaccard, lemma_jaccard_distance*

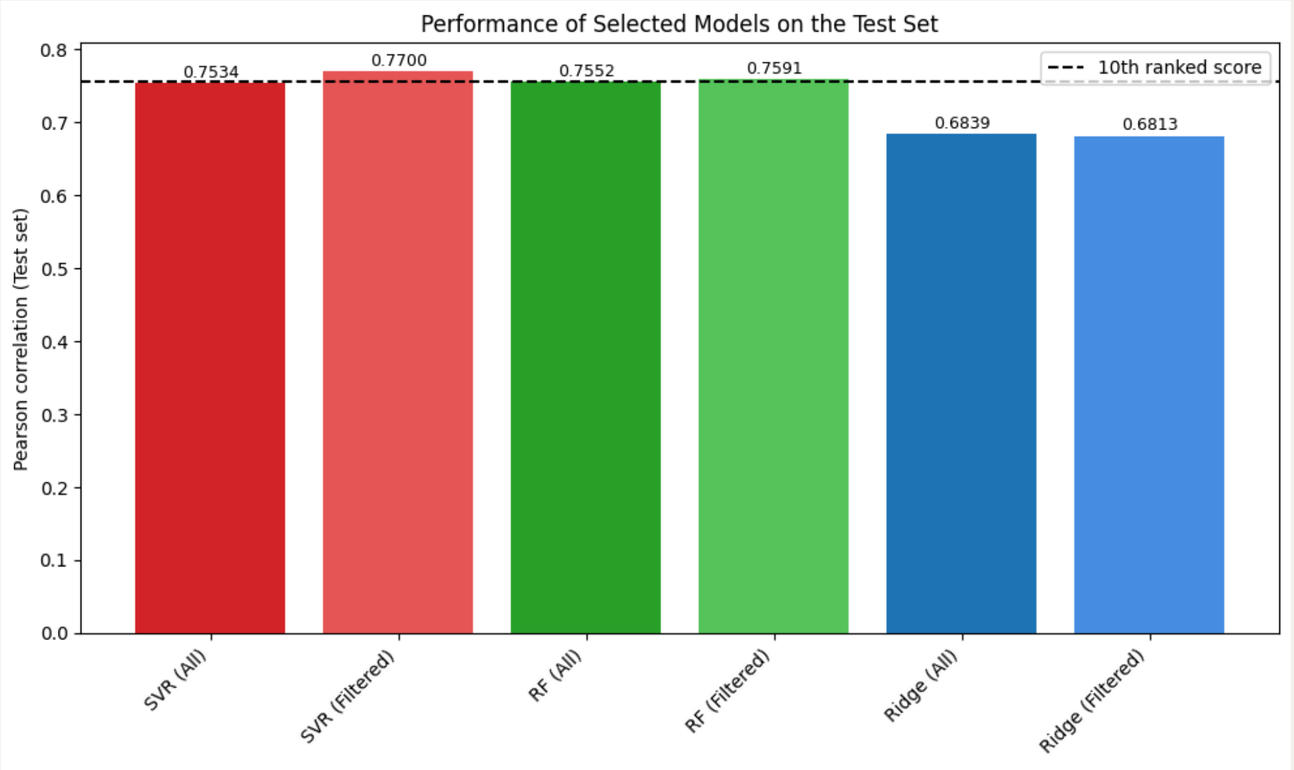
**POS-based similarity:** *pos_jaccard_distance*

**Greedy String tiling:** *greedy_string_tiling*

# Results on Test Set

## On the whole test set

- **SVR and Random Forests** still on top
- All models **lower than their validation** counterpart (expected but see more in depth next slide)
- the filtered version generally outperforms the version with all features
  - especially SVR
  - confirms **generalization** ability of our model



# Results on Test Set

## Best three models performance on individual datasets

- **High variation in performance** depending on the dataset
- Very **high on MSR vid**, similar to validation metric
- Decent on MSRPAR and OnWN
- **Subpar on SMTeuroparl and SMTnews**
- The 3 models performance vary similarily across datasets.

| Dataset | Size | RF (All) | RF (Filtered) | SVR (Filtered) |
|---|---|---|---|---|
| **MSRpar** | 750 | 0.609446 | 0.615545 | 0.628061 |
| **MSRvid** | 750 | 0.864511 | 0.864889 | 0.856012 |
| **SMTeuroparl** | 459 | 0.500213 | 0.516677 | 0.46873 |
| **surprise.OnWN** | 750 | 0.645688 | 0.651757 | 0.613276 |
| **surprise.SMTnews** | 399 | 0.504856 | 0.48512 | 0.487964 |

# Analysis takeaways

- Simple, well-designed linguistic features can still capture a surprising amount of semantic structure, especially when matched **with non-linear models.**
- **Combining diverse feature families** (lexical, syntactic) > complementary gains
- Feature elimination showed **strong redundancy among many features**. need for well-crafted non-redundant features
- Error analysis revealed weaknesses: **WordNet semantic distances often diverge from human intuition**
- Features make the model explainable but struggle to model context elegantly (compared to embeddings)

# REFERENCES

[1] Sarić, F., Glavaš, G., Karan, M., Šnajder, J., & Dalbelo Bašić, B. (2012). TakeLab: Systems for Measuring Semantic Text Similarity. University of Zagreb, FER.
[2] Bar, D., Biemann, C., Gurevych, I., & Zesch, T. (2012). UKP: Computing Semantic Textual Similarity by Combining Multiple Content Similarity Measures. UKP-TUDA & UKP-DIPF, Technische Universität Darmstadt and DIPF.

# THANK YOU!